

A Bit-Parallel Algorithm to Solve the Nearest Neighbor String Matching Problem

Chia Shin Ou and R. C. T. Lee
Department of Computer Science
National Tsing Hua University, Taiwan
Department of Computer Science

d9662836@oz.nthu.edu.tw and rctlee@ncnu.edu.tw

Abstract

We present an efficient algorithm to solve the nearest neighbor string matching problem. The nearest neighbor string matching problem has useful applications in bioinformatics. We build on a previous technique which is used to solve the approximate string matching problem. They proposed an idea which combines a bit-parallel approach and a filter rule to speed up the traditional approach of approximate string matching problem. The bit-parallel approach is based on a bit-parallel version of dynamic programming (DP) method which is proposed by Myers. And, the filter rule is to find a substring S whose edit distance between S and any substring of pattern P is larger than an error bound k . If S is found, we can safely move P to decrease some unnecessary operations. To use this idea, we can solve the nearest neighbor string matching problem more efficiently.

1 Introduction

The nearest neighbor string matching problem is one of the main string matching problems such as exact string matching problem[3,6,13], approximate string matching[1,7,14-20], longest common subsequence problem[8,9,12], multiple pattern string matching problem[5], and longest tandem repeat subsequence problem[2,4]. The nearest neighbor string matching problem also has application to text searching, computation biology and so on. The nearest neighbor string matching problem is defined as follows: Given a text string $T = t_1 t_2 \dots t_n$ and a pattern string $P = p_1 p_2 \dots p_m$, find all substrings of T whose edit distances with P are the smallest. To be more precise, we are looking for all positions i of T such that there exists a suffix of $T(1, i)$ whose edit distance with P is the smallest. Such suffixes are called nearest neighbors of P in T . Throughout the paper, given a string $A = a_1 a_2 \dots a_m$,

we denote $A(i, j)$ to be the substring $a_i a_{i+1} \dots a_j$. Given strings A and B , we denote $ED(A, B)$ to be the edit distance between A and B . This problem can be computed by using the DP method which is used by [18] to solve the approximate string matching problem. The approximate string matching problem is defined as follows: Given a text string $T = t_1 t_2 \dots t_n$ and a pattern string $P = p_1 p_2 \dots p_m$, for each location i in text T , find the suffix S_i of $T(1, i)$ such that $ED(S_i, P)$ is the smallest. We are also asked to find $ED(S_i, P)$. To solve the problem, we further define S_{ij} to be the suffix of $T(1, i)$ and S_{ij} such that $ED(S_{ij}, P(1, j))$ is the shortest. To simplify the notation, we let $c(i, j)$ denote $ED(S_{ij}, P(1, j))$. Then, $c(i, j)$ can be computed by the following formulas:

$$\begin{aligned} c(i, 0) &= 0, \quad c(0, j) = j, \\ c(i, j) &= c(i-1, j-1) \\ &\text{if } (p_i = w_j) \text{ else } 1 + \min(c(i-1, j), c(i, j-1), c(i-1, j-1)) \end{aligned}$$

In this paper, we are interested in mechanisms to compute the above table efficiently by using bit-parallel approach[10,11,14,16,17]. Our approach is based upon the bit-parallel algorithm presented by Heikki Hyyro and Gozalo Navarro[11] which was designed to solve the following type of approximate string matching problem: Given a text string T and a pattern string P , for each location i , determine whether there exists a substring ending at location i whose edit distance with P is less than or equal to a prepecified error bound k .

2 HN Bit-Parallel Algorithm

Heikki Hyyro and Gozalo Navarro's algorithm (HN algorithm)[11] is to fill a DP matrix by using the bit-parallel approach which is based on Myers' original bit-parallel method[14].

Formally, define the horizontal value $\Delta h(i, j) = c(i, j) - c(i, j-1)$, the vertical value $\Delta v(i, j) = c(i, j) - c(i-1, j)$ and diagonal value

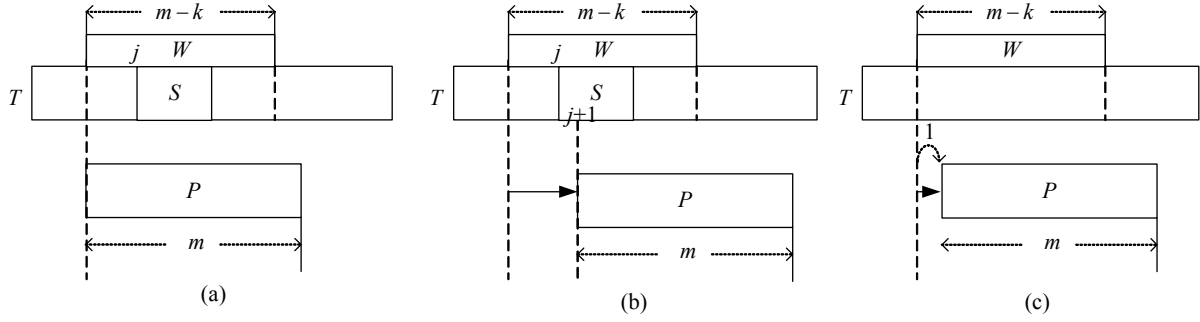


Figure 1

$\Delta d(i,j) = c(i,j) - c(i-1,j-1)$. According to [15,19], we have the following lemma:

Lemma 1. [15,19] For all i, j : $\Delta v(i, j), \Delta h(i, j) \in \{-1, 0, 1\}$ and $\Delta d(i, j) \in \{0, 1\}$.

Let us consider the following boolean variables:

$$\begin{aligned} VP(i, j) &\equiv \Delta v(i, j) = +1, & VN(i, j) &\equiv \Delta v(i, j) = -1, \\ HP(i, j) &\equiv \Delta h(i, j) = +1, & HN(i, j) &\equiv \Delta h(i, j) = -1, \\ D0(i, j) &\equiv \Delta d(i, j) = 0. \end{aligned}$$

The matrices HN, VN, HP, VP , and $D0$ can be seen as vectors indexed by i , which change their value for each new text position j , as we traverse the text. These matrices can be computed by the following algorithm.

Algorithm 1: MyersStep(w_i)

```

 $X \leftarrow w_i \mid VN$ 
 $D0 \leftarrow ((VP + (X \& VP)) \wedge VP) \mid X$ 
 $HN \leftarrow VP \& D0$ 
 $HP \leftarrow VN \mid (VP \mid D0)$ 
 $X \leftarrow (HP \ll 1) \mid 0^{m-1}$ 
 $VN \leftarrow X \& D0$ 
 $VP \leftarrow (HN \ll 1) \mid (X \mid D0)$ 

```

The most important value in the DP matrix is the last value $c(m, j)$ in the column j of the DP matrix. The solutions can be determined by checking whether $c(m, j) \leq k$ for all j or not. Hence, the value differ stores $c(m, j)$ and is updated by using $HP(m, j)$ and $HN(m, j)$. According to Myers' bit-parallel algorithm, we can use the following algorithm to compute the whole DP matrix column by column.

The greedy method is to fill the whole DP matrix. In [11], authors use a filter approach to determine and avoid the impossible regions in the text T by using the diagonal property [13,19] which is shown in the following.

Lemma 2. (Diagonal property) For all i and j ,

values $c(i, j)$ of DP matrix in the diagonal d where $d = i - j$ are non-decreasing.

This lemma allows us to know the largest l such that $c(i, l) \leq k$ for each i . Clearly l exists if and only if $c(i, j) \leq k$ for some j . Hence, we will maintain a witness in the current column i of the DP matrix that will tell us the last cell which does not exceed k . Initially, since $c(0, j) = 0$, we set $l = k$. Since $c(i, l+1) - c(i-1, l) = \Delta d(i, l+1) \in \{0, 1\}$, l is increased by 1 if and only if the bit $D0(i, l) = 1$. If $D0(i, l) = 0$, l will be kept to be the same or decreased. If we reach $l = 0$ and still $c(i, l) > k$, then all the rows are larger than k and we stop the scanning process. If $l = m$, we have $c(i, m) \leq k$ and found an occurrence at location i in T . According to lemma 2, we have the following algorithm.

Algorithm 2: Myers' Bit-Parallel Algorithm

```

 $VP \leftarrow 1^m, VN \leftarrow 0^m$ 
 $l \leftarrow 0^{m-k} 10^{k-1}$ 
While  $j \leq n$  Do
  BPMStep( $w_i$ )
   $l \leftarrow l \ll 1$ 
  If  $D0 \& l = 0^m$  Then
     $val \leftarrow k + 1$ 
    While  $val > k$  Do
      IF  $l = 0^{m-1} 1$  Then Return FALSE
      IF  $VP \& l \neq 0^m$ 
        Then  $val \leftarrow val - 1$ 
      IF  $VN \& l \neq 0^m$ 
        Then  $val \leftarrow val + 1$ 
       $l \leftarrow l \gg 1$ 
    Else If  $l = 10^{m-1}$ 
      Then Return TRUE
     $j \leftarrow j + 1$ 
Return FALSE

```

In order to avoid searching all positions of occurrences in T , [11] proposed the second filter according to the following lemma.

Lemma 3. [11, 13] Given a partial window W in text T with length $m-k$. If there exists a substring S at the position j of T in W such that the edit

distance between S and any substring of P is larger than k , then we can safely move P to the position $j+1$ of T . Otherwise, we shift P by one step.

According to the lemma 3, if a substring S at the position j of T such that the edit distance between S and any substring of P is larger than k , it means the values in column j of DP matrix are larger than k . [11] uses a backward scanning algorithm which indicates, as early as possible, that there exists a substring S such that no substring P' in P has the property that $ED(S, P') \leq k$. Thus, we can shift the window as early as possible. Because it is difficult to check all values in the column j of DP matrix, [10] proposed an idea whether some witness values in the column j of DP matrix are larger than k' or not, and these witness values have the same interval distance Q . Let k' be $k + \lfloor Q/2 \rfloor$. Let $t = \lceil m/Q \rceil$ be consecutive witnesses into vector MC . In order to determine Q , we have the following lemma.

Lemma 3. [11] : $Q = \lceil \log_2(m-k+1) \rceil$ if $m-k-k' \leq 2^{Q-1}$ and $k'+1 \leq 2^{Q-1}$; otherwise, $Q+1$.

For updating all of the witness values, each witness $c(m-rQ, j)$ can be updated by considering the $(m-rQ)$ -th bits of HP and HN . That is, it defined a mask $sMask = (0^{Q-1})^t 0^{m+Q-1-tQ}$ and update all witnesses in parallel by the following formula:

$$MC = MC + (HP \& sMask) - (HN \& sMask).$$

For determining that all witnesses have exceed k' , we store each witness with extra value $b = 2^{Q-1} - 1 - k'$. That is, if $c(m-rQ, j) = x$, the corresponding witness store the value $x+b$. Hence, $MC = [b]_0^{t-1} 0^{m+Q-1-tQ}$ initially. By this way, if the Q -th bit of a witness is equal to 1, the corresponding value in DP matrix is larger than k' . Hence, it defined $eMask = (10^{Q-1})^t 0^{m+Q-1-tQ}$ to check whether the Q -th bits of all witnesses are equal to 1 or not by using the following formula:

$$MC \& eMask = eMask$$

Thus, it can stop the scanning of W whenever $MC \& eMask = eMask$. The HN Bit-Parallel Algorithm is shown in the following:

Algorithm 3 : HN Bit-Parallel Algorithm

Preprocessing

For $c \in \Sigma$ Do $Bf[c] \leftarrow 0^m, Bb[c] \leftarrow 0^m$

```

For  $i \in 1 \dots m$  Do
   $Bf[P_i] \leftarrow Bf[P_i] \mid 0^{m-i} 10^{i-1}$ 
   $Bb[P_i] \leftarrow Bb[P_i] \mid 0^{i-1} 10^{m-i}$ 
   $Q \leftarrow \lceil \log_2(m-k+1) \rceil$ 
  If  $2^{Q-1} < \max(m-2k - \lfloor Q/2 \rfloor, k+1 + \lfloor Q/2 \rfloor)$ 
    Then  $Q \leftarrow Q+1$ 
   $b \leftarrow 2^{Q-1} - k - \lfloor Q/2 \rfloor - 1$ 
   $t \leftarrow \lceil m/Q \rceil$ 
   $sMask \leftarrow (0^{Q-1})^t 0^{m+Q-1-tQ}$ 
   $eMask \leftarrow (10^{Q-1})^t 0^{m+Q-1-tQ}$ 

```

Searching

```

pos  $\leftarrow 0$ 
While pos  $\leq n - (m-k)$  Do
   $j \leftarrow m-k, last \leftarrow m-k$ 
   $VP \leftarrow 0^m, VN \leftarrow 0^m$ 
   $MC \leftarrow [b]_0^{t-1} 0^{m+Q-1-tQ}$ 
  While  $j \neq 0$  AND  $MC \& eMask \neq eMask$  Do
    MyersStep( $Bb[T_{pos+j}]$ )
     $MC \leftarrow MC + (HP \& sMask) - (HN \& sMask)$ 
     $j \leftarrow j-1$ 
    If  $MC \& 10^{m+Q-2} = 0^{m+Q-1}$  Then
      If  $j > 0$  Then  $last \leftarrow j$ 
    Else If Myers Algorithm( $Bf, T_{pos+1 \dots n}$ ) Then
      Report an occurrence at pos + 1
  pos  $\leftarrow pos + last$ 

```

3 Finding the Nearest Neighbors

In this section, we present a bit-parallel approach based on [14]. It is more efficient than traditional dynamic programming approach [18]. Since definition of the nearest neighbor string matching problem without giving the error bound k , we can not solve the nearest neighbor string matching problem by using [11]'s bit-parallel method. Hence, we propose a variant version of bit-parallel method to solve the nearest neighbor string matching problem.

In our algorithm, there are two phases. In the first phases, we build a floating window FW with size $m+nd$ where $nd=m$ because the distance of the nearest neighbor is m at the beginning. If we find the nearest neighbor with smaller distance sd , we can reduce the size of FW to $m+sd$ where $sd < nd$ safely and shift the pattern P by one step in this phase.

For instance, let $T=actgtac$ and $P=acc$. First, the size of FW is $m+nd=6$ where $nd=m=3$ which is shown in Figure 2.

	1	2	3	4	5	6	7
T	FW						
	a	c	t	g	t	a	c
P	1	2	3				
	a	c	t				

Figure 2

By computing the 1st column of DP matrix, we find the nearest neighbor with smaller distance $sd=2$. Since, we can reduce $|FW|=5$ which is shown in Figure 3.

		1	2	3	4	5	6	7
	T	a	c	t	g	t	a	c
P	0	1	2	3	4	5	6	7
1	a	1	0					
2	c	2	1					
3	c	3	2					

	1	2	3	4	5	6	7
T	a	c	t	g	t	a	c
P	1	2	3				
	a	c	t				

Figure3

By computing the 2nd column of DP matrix, we find the nearest neighbor with smaller distance $sd=1$. Since, we can reduce $|FW|=4$ which is shown in Figure 4.

		1	2	3	4	5	6	7
	T	a	c	t	g	t	a	c
P	0	1	2	3	4	5	6	7
1	a	1	0	1				
2	c	2	1	0				
3	c	3	2	1				

		1	2	3	4	5	6	7
	T	a	c	t	g	t	a	c
P	1	2	3					
		a	c	t				

Figure 4

Similarly, we can continue to compute the DP

matrix until reaching the size of FW which is shown in the Figure 5. And, we shift P by one step.

		1	2	3	4	5	6	7
	T	a	c	t	g	t	a	c
P	0	1	2	3	4	5	6	7
1	a	1	0	1	2			
2	c	2	1	0	1	2		
3	c	3	2	1	1	2		

	1	2	3	4	5	6	7
T	a	c	t	g	t	a	c
P	1	2	3				
	a	c	t				

--->

	1	2	3
P	a	c	t

Figure 5

In this phase, the DP matrix can be computed by using Myers' algorithm[14] with some modifications. We show the algorithm in the following:

Algorithm 4: Phase 1 of Finding the Nearest Neighbors Algorithm

$VP \leftarrow 1^m, VN \leftarrow 0^m$
 $l \leftarrow 10^{m-1}$
 $nd \leftarrow m$
 $sd \leftarrow m$
 While $j \leq m+nd$ Do
 BPMStep(w_j)
 IF $VP \& l \neq 0^m$ Then $sd \leftarrow sd - 1$
 IF $VN \& l \neq 0^m$ Then $sd \leftarrow sd + 1$
 If $sd < nd$ Then $nd \leftarrow sd$
 $j \leftarrow j + 1$

In the phase 2, our algorithm is based on Heikki Hyyro and Gozalo Navarro's algorithm. Here, we build on a floating window W with length $m+nd'$ where nd' is the length of the nearest neighbor in the phase 1. The difference between our algorithm and Heikki Hyyro and Gozalo Navarro's algorithm is that the witness's size Q is fixed by the error bound k in HN's algorithm. In our algorithm, the length of witness is floating according to the edit distance of the present nearest neighbors sd in the last time of window comparison. In our algorithm, we set Q by the following formula:

$Q = \lceil \log_2(m - sd + 1) \rceil$ if $m - sd - sd' \leq 2^{Q-1}$ and $sd' + 1 \leq 2^{Q-1}$ where $sd' = sd + \lfloor Q/2 \rfloor$; otherwise, $Q = Q + 1$.

And, we set the extra value $b = 2^{Q-1} - 1 - sd'$. For instance, Given $T = agattaagctgata$ and $P = agagct$. After the phase 1, we shift P by on step and get $sd=2$. In phase 2, we set $Q = \lceil \log_2(6-2+1) \rceil = 3$, $t = \lceil 6/3 \rceil = 2$, $sMask = 00100100$, $eMask = 10010000$, $sd' = 2 + \lfloor 3/2 \rfloor = 3$ and $b = 2^2 - 1 - 3 = 0$. And, we open a window $W = gattaagc$. After reading $w_1 = 'g'$, we have $HP=000000$ and $HN=011111$. $MC = 01111000 + (000000 \& 00100100) - (011111 \& 00100100) = 01010100$. After reading $w_2 = 'a'$, we have $HP=010000$ and $HN=001111$. $MC = 01010100 + (010000 \& 00100100) - (001111 \& 00100100) = 00110000$. After reading $w_3 = 't'$, we have $HP=101000$ and $HN=000001$. $MC = 00110000 + (101000 \& 00100100) - (000001 \& 00100100) = 01101100$. After reading the next $w_4 = 't'$, we have $HP=111100$ and $HN=000000$. $MC = 01101100 + (111100 \& 00100100) - (000000 \& 00100100) = 10001100$. After reading the next $w_4 = 'a'$, we have $HP=111110$ and $HN=000001$. $MC = 10001100 + (111110 \& 00100100) - (000001 \& 00100100) = 10110000$. In this stage, $eMask \& MC = eMask$. Hence, we can shift P to match with w_7 safely. And, we do not find the smaller nearest neighbor in this stage. In the next stage, we still open a new window with length 8.

In the following, we show the phase 2 of finding the nearest neighbor string matching algorithm.

Algorithm 5: Phase 2 of Finding the Nearest Neighbors Algorithm

Preprocessing

For $c \in \Sigma$ **Do** $Bf[c] \leftarrow 0^m$, $Bb[c] \leftarrow 0^m$

For $i \in 1 \dots m$ **Do**

$Bf[Pi] \leftarrow Bf[Pi] \mid 0^{m-i}10^{i-1}$

$Bb[Pi] \leftarrow Bb[Pi] \mid 0^{i-1}10^{m-i}$

$Q \leftarrow \lceil \log_2(m - sd + 1) \rceil$

If $2^{Q-1} < \max(m - 2sd - \lfloor Q/2 \rfloor, sd + 1 + \lfloor Q/2 \rfloor)$

Then $Q \leftarrow Q + 1$

$b \leftarrow 2^{Q-1} - sd - \lfloor Q/2 \rfloor - 1$

$t \leftarrow \lfloor m/Q \rfloor$

$sMask \leftarrow (0^{Q-1}1)^t 0^{m+Q-1-tQ}$

$eMask \leftarrow (10^{Q-1})^t 0^{m+Q-1-tQ}$

Searching

$pos \leftarrow 0$

While $pos \leq n - (m - sd)$ **Do**

$j \leftarrow m - sd$, $last \leftarrow m - sd$

$VP \leftarrow 0^m$, $VN \leftarrow 0^m$

$MC \leftarrow [b]_Q^t 0^{m+Q-1-tQ}$

While $j \neq 0$ **AND** $MC \& eMask \neq eMask$ **Do**

MyersStep($Bb[T_{pos+j}]$)

$MC \leftarrow MC + (HP \& sMask) - (HN \& sMask)$

$j \leftarrow j - 1$

If $MC \& 10^{m+Q-2} = 0^{m+Q-1}$ **Then**

If $j > 0$ **Then** $last \leftarrow j$

Else If **Myers Algorithm**($Bf, T_{pos+1 \dots n}$) **Then**

Store an occurrence at $pos + 1$

$pos \leftarrow pos + last$

4 Conclusion

In this paper, we use the bit-parallel approach to solve the nearest neighbor problem. The tradition way to solve this problem is to construct the DP matrix in all locations of T . This approach is greedy and inefficiency. By using bit-parallel approach and the filter methods, we can speed up the tradition DP matrix approach to avoid exhaustive searching in the text. The nearest neighbor string matching problem also has useful applications in bioinformatics.

References

- [1] A. Amir, D. Keselman, G. M. Landau, M. Lewenstein, N. Lewenstein and M. Rodeh. Text Indexing and Dictionary Matching with One Error, *Journal of Algorithms*, 37:309-325, 2000.
- [2] G. Benson. Tandem Repeats Finder: a Program to Analyze DNA Sequences, *Nucleic Acids Research*, 27(2):573-580, 1999.
- [3] R. Baeza-Yates and G. H. Gonnet. A new approach to text searching *Commun. Journal of the ACM*, 35:74-82, 1992.
- [4] M. Buchner and S. Janjarasjitt. Detection and Visualization of Tandem Repeats in DNA Sequences, *IEEE Transactions on Signal Processing*, 51:2280-2287, 2003.
- [5] K. Fredriksson and G. Navarro. Average-Optimal Single and Multiple Approximate String Matching, *ACM Journal of Experimental Algorithmics*, 9:1-47, 2004.
- [6] D. Gusfield. Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, *Textbook*, Cambridge University Press, 1997.
- [7] G. Landau and U. Vishkin. Fast parallel and serial approximate string matching, *Journal of Algorithms*, 10: 157-169, 1989.
- [8] D. S. Hirschberg. A Linear Space Algorithm for Computing Maximal Common Subsequences, *Communications of the ACM*, Vol.18, 6:341-343, 1975.
- [9] D. S. Hirschberg. Algorithms for the longest common subsequence problem, *Journal of the ACM*, 24:664-675, 1977.
- [10] H. Hyiro. Bit-parallel approximation string matching algorithms with transposition, *Journal of Discrete Algorithms*, 3:215-229,

- 2005.
- [11] H. Hyryro and G. Navarro. Bit-parallel Witnesses and their Application to Approximate String Matching, *Algorithmica*, 41(3):203-231, 2005.
 - [12] J. W. Hunt and T. G. Szymanski. A fast Algorithm for Computing Longest Common Subsequences, *Communications of the ACM*, 20(5):350-353, 1977.
 - [13] R. C. T. Lee. String Matching Algorithm, *Textbook*, 2010.
 - [14] G. Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming, *Journal of the ACM*, 46(3):395-415, 1999.
 - [15] W. J. Masek and M. S. Paterson. A faster algorithm computing string edit distances, *Journal of Computer and System Sciences*, 20:18-31, 1980.
 - [16] C. S. Ou and R. C. T. Lee, A Parallel Approach to Solve the Approximation String Matching Problem, *Proceedings of the Workshop on Combinatorial Mathematics and Computation Theory*, 2010.
 - [17] C. S. Ou, C. L. Lu and R. C. T. Lee. A Bit-Parallel Approach to Solve Approximation String Matching Problem for Unlimited Pattern Length, *Asian Association for Algorithms and Computation*, 2011.
 - [18] P. H. Sellers. The theory and computation of evolutionary distances: pattern recognition, *Journal of Algorithms*, 1:359-373, 1980.
 - [19] E. Ukkonen. Finding Approximate patterns in strings. *Journal of Algorithms*, 6: 100-118 1985.
 - [21] S. Wu and U. Manber. Fast Text Searching: Allowing Errors, *Communications of the ACM*, 35:83-91, 1992.
 - [22] Y. Wexler, Z. Yakhini, Y. Kashi and D. Geiger. Finding Approximate Tandem Repeats in Genomic Sequences, *Proceedings of the eighth annual international conference on Computational molecular biology*, 2004.